Experiment 29: VAE/GAN Sample Generation

Question

Problem Statement: A fashion retailer wants to generate new product images to prototype designs before manufacturing. The Fashion-MNIST dataset (clothing and footwear images, 28×28). The objective is to build a VAE and a GAN and generate new samples. Use TensorFlow, Keras, NumPy, Matplotlib.

Tasks:

1. Build a VAE with encoder, reparameterisation and decoder; train with reconstruction plus KL loss.
2. Generate images by sampling the latent space; plot a 2-D latent-space grid.
3. Build a DCGAN with a generator and a discriminator.
4. Train the GAN and save generated images every few epochs.
5. Plot generator and discriminator loss curves.

Code

```
"""Experiment 29: VAE / GAN Sample Generation (Fashion-MNIST)"""
import os
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers


np.random.seed(42); tf.random.set_seed(42)
EPOCHS_VAE, EPOCHS_GAN, BATCH = 15, 20, 128
LATENT_VAE, LATENT_GAN = 2, 100              # 2-D latent space lets us plot a latent grid
SAVE_EVERY = 5                               # save GAN sample images every 5 epochs
os.makedirs("gan_samples", exist_ok=True)


# ---------------- Data: Fashion-MNIST (28x28 grayscale) ----------------
CLASS_NAMES = ["T-shirt", "Trouser", "Pullover", "Dress", "Coat", "Sandal", "Shirt", "Sneaker", "Bag", "Ankle
boot"]
def load_fashion():
    try:
        (x, y), _ = keras.datasets.fashion_mnist.load_data()
        return x[..., None].astype("float32") / 255.0
    except Exception as e:                                          # offline stand-in so that the code
can still run
        print("WARNING: Fashion-MNIST not available (", type(e).__name__, ") -> using scikit-learn digits as
a stand-in.")
        from sklearn.datasets import load_digits
        from scipy.ndimage import zoom
        d = np.stack([zoom(im / 16.0, 3.5, order=1) for im in load_digits().images])
        return np.clip(d, 0, 1)[..., None].astype("float32")


x_train = load_fashion()
print("Training images:", x_train.shape)
ds01 = tf.data.Dataset.from_tensor_slices(x_train).shuffle(10000).batch(BATCH, drop_remainder=True)       #
[0,1] for the VAE
ds11 = tf.data.Dataset.from_tensor_slices(x_train * 2 - 1).shuffle(10000).batch(BATCH, drop_remainder=True)
# [-1,1] for the GAN


def show_grid(images, n_rows, n_cols, title, path=None):
    fig, axes = plt.subplots(n_rows, n_cols, figsize=(n_cols * 1.2, n_rows * 1.2))
    for a, im in zip(axes.ravel(), images):
        a.imshow(np.squeeze(im), cmap="gray"); a.axis("off")
    plt.suptitle(title); plt.tight_layout()
    if path: plt.savefig(path, dpi=80)
    plt.show()


# ================================ VAE ================================
# ---------------- Task 1: encoder, reparameterisation, decoder ----------------
enc_in = keras.Input((28, 28, 1))
x = layers.Conv2D(32, 3, strides=2, padding="same", activation="relu")(enc_in)
x = layers.Conv2D(64, 3, strides=2, padding="same", activation="relu")(x)
x = layers.Dense(128, activation="relu")(layers.Flatten()(x))
```

```
z_mean = layers.Dense(LATENT_VAE, name="z_mean")(x)
z_log_var = layers.Dense(LATENT_VAE, name="z_log_var")(x)
encoder = keras.Model(enc_in, [z_mean, z_log_var], name="encoder")


dec_in = keras.Input((LATENT_VAE,))
y = layers.Dense(7 * 7 * 64, activation="relu")(dec_in)
y = layers.Reshape((7, 7, 64))(y)
y = layers.Conv2DTranspose(64, 3, strides=2, padding="same", activation="relu")(y)
y = layers.Conv2DTranspose(32, 3, strides=2, padding="same", activation="relu")(y)
dec_out = layers.Conv2DTranspose(1, 3, padding="same", activation="sigmoid")(y)
decoder = keras.Model(dec_in, dec_out, name="decoder")


vae_opt = keras.optimizers.Adam(1e-3)


@tf.function
def vae_step(batch):
    with tf.GradientTape() as tape:
        mu, log_var = encoder(batch, training=True)
        eps = tf.random.normal(tf.shape(mu))
        z = mu + tf.exp(0.5 * log_var) * eps                                    # reparameterisation trick
        recon = decoder(z, training=True)
        rec_loss = tf.reduce_mean(tf.reduce_sum(keras.losses.binary_crossentropy(batch, recon), axis=(1, 2)))
        kl = tf.reduce_mean(-0.5 * tf.reduce_sum(1 + log_var - tf.square(mu) - tf.exp(log_var), axis=1))
        loss = rec_loss + kl                                                    # reconstruction + KL
    vars_ = encoder.trainable_variables + decoder.trainable_variables
    vae_opt.apply_gradients(zip(tape.gradient(loss, vars_), vars_))
    return loss, rec_loss, kl


vae_hist = []
for ep in range(1, EPOCHS_VAE + 1):
    vals = np.array([[float(t) for t in vae_step(b)] for b in ds01]).mean(axis=0)
    vae_hist.append(vals)
    print(f"VAE epoch {ep:2d}: total={vals[0]:.2f}  reconstruction={vals[1]:.2f}  KL={vals[2]:.2f}")
vae_hist = np.array(vae_hist)
plt.figure(figsize=(7, 4)); plt.plot(vae_hist[:, 0], label="total"); plt.plot(vae_hist[:, 1],
label="reconstruction"); plt.plot(vae_hist[:, 2], label="KL")
plt.xlabel("Epoch"); plt.title("VAE loss"); plt.legend(); plt.show()


# ---------------- Task 2: sample the latent space + 2-D latent grid ----------------
samples = decoder(tf.random.normal((16, LATENT_VAE))).numpy()
show_grid(samples, 2, 8, "VAE: images generated from z ~ N(0, I)")


n = 15; grid = np.linspace(-3, 3, n)
canvas = np.zeros((28 * n, 28 * n))
for i, yi in enumerate(grid[::-1]):
    for j, xj in enumerate(grid):
        canvas[i * 28:(i + 1) * 28, j * 28:(j + 1) * 28] = decoder(np.array([[xj, yi]],
dtype="float32")).numpy()[0, :, :, 0]
plt.figure(figsize=(8, 8)); plt.imshow(canvas, cmap="gray"); plt.axis("off")
plt.title("VAE latent-space grid (z1 horizontal, z2 vertical, range -3..3)"); plt.show()
decoder.save("vae_decoder.keras")


# ============================== DCGAN ==============================
# ---------------- Task 3: generator and discriminator ----------------
generator = keras.Sequential([
    layers.Input((LATENT_GAN,)),
    layers.Dense(7 * 7 * 128, use_bias=False), layers.BatchNormalization(), layers.LeakyReLU(0.2),
layers.Reshape((7, 7, 128)),
    layers.Conv2DTranspose(64, 5, strides=1, padding="same", use_bias=False), layers.BatchNormalization(),
layers.LeakyReLU(0.2),
    layers.Conv2DTranspose(32, 5, strides=2, padding="same", use_bias=False), layers.BatchNormalization(),
layers.LeakyReLU(0.2),
    layers.Conv2DTranspose(1, 5, strides=2, padding="same", activation="tanh"),
], name="generator")
discriminator = keras.Sequential([
    layers.Input((28, 28, 1)),
    layers.Conv2D(64, 5, strides=2, padding="same"), layers.LeakyReLU(0.2), layers.Dropout(0.3),
    layers.Conv2D(128, 5, strides=2, padding="same"), layers.LeakyReLU(0.2), layers.Dropout(0.3),
    layers.Flatten(), layers.Dense(1),
], name="discriminator")


bce = keras.losses.BinaryCrossentropy(from_logits=True)
g_opt = keras.optimizers.Adam(2e-4, beta_1=0.5); d_opt = keras.optimizers.Adam(2e-4, beta_1=0.5)
```

```
@tf.function
def gan_step(real):
    noise = tf.random.normal((tf.shape(real)[0], LATENT_GAN))
    with tf.GradientTape() as gt, tf.GradientTape() as dt:
        fake = generator(noise, training=True)
        real_out, fake_out = discriminator(real, training=True), discriminator(fake, training=True)
        g_loss = bce(tf.ones_like(fake_out), fake_out)                                    # generator
wants D(fake) = 1
        d_loss = bce(tf.ones_like(real_out), real_out) + bce(tf.zeros_like(fake_out), fake_out)
    g_opt.apply_gradients(zip(gt.gradient(g_loss, generator.trainable_variables),
generator.trainable_variables))
    d_opt.apply_gradients(zip(dt.gradient(d_loss, discriminator.trainable_variables),
discriminator.trainable_variables))
    return g_loss, d_loss

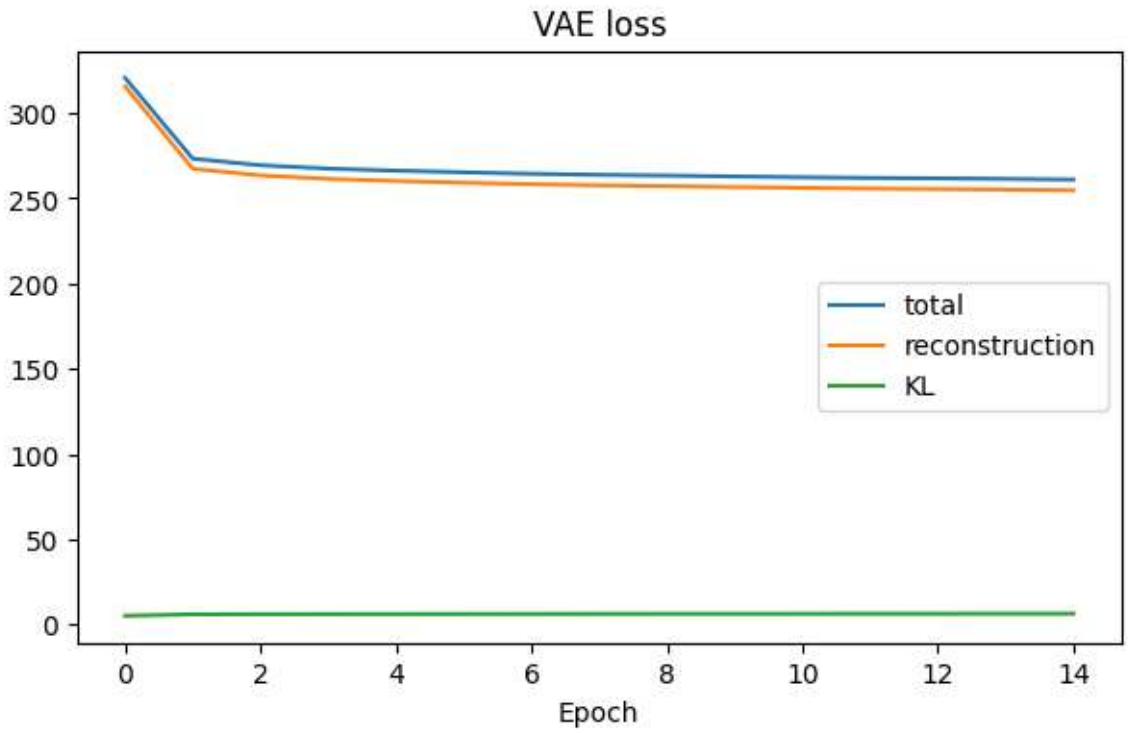

# ---------------- Task 4: train and save generated images every few epochs ----------------
fixed_noise = tf.random.normal((16, LATENT_GAN))
gan_hist = []
for ep in range(1, EPOCHS_GAN + 1):
    vals = np.array([[float(t) for t in gan_step(b)] for b in ds11]).mean(axis=0)
    gan_hist.append(vals)
    print(f"GAN epoch {ep:2d}: generator loss={vals[0]:.3f}  discriminator loss={vals[1]:.3f}")
    if ep % SAVE_EVERY == 0 or ep == 1:
        imgs = (generator(fixed_noise, training=False).numpy() + 1) / 2
        show_grid(imgs, 2, 8, f"GAN samples after epoch {ep}", path=f"gan_samples/epoch_{ep:03d}.png")
gan_hist = np.array(gan_hist)
generator.save("gan_generator.keras")


# ---------------- Task 5: loss curves ----------------
plt.figure(figsize=(7, 4)); plt.plot(gan_hist[:, 0], label="Generator loss"); plt.plot(gan_hist[:, 1],
label="Discriminator loss")
plt.xlabel("Epoch"); plt.title("DCGAN training losses"); plt.legend(); plt.grid(alpha=0.3); plt.show()

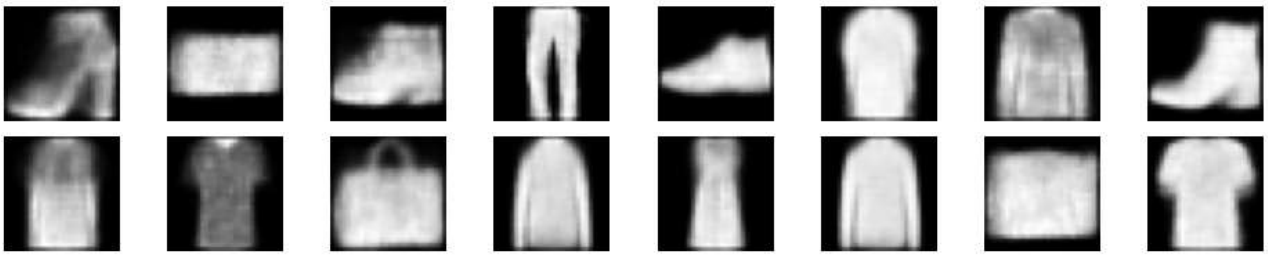

fake = (generator(tf.random.normal((16, LATENT_GAN)), training=False).numpy() + 1) / 2
show_grid(fake, 2, 8, "Final DCGAN samples")
print("Saved: vae_decoder.keras, gan_generator.keras and gan_samples/*.png  (Experiment 30 uses the two
models)")
```

Output

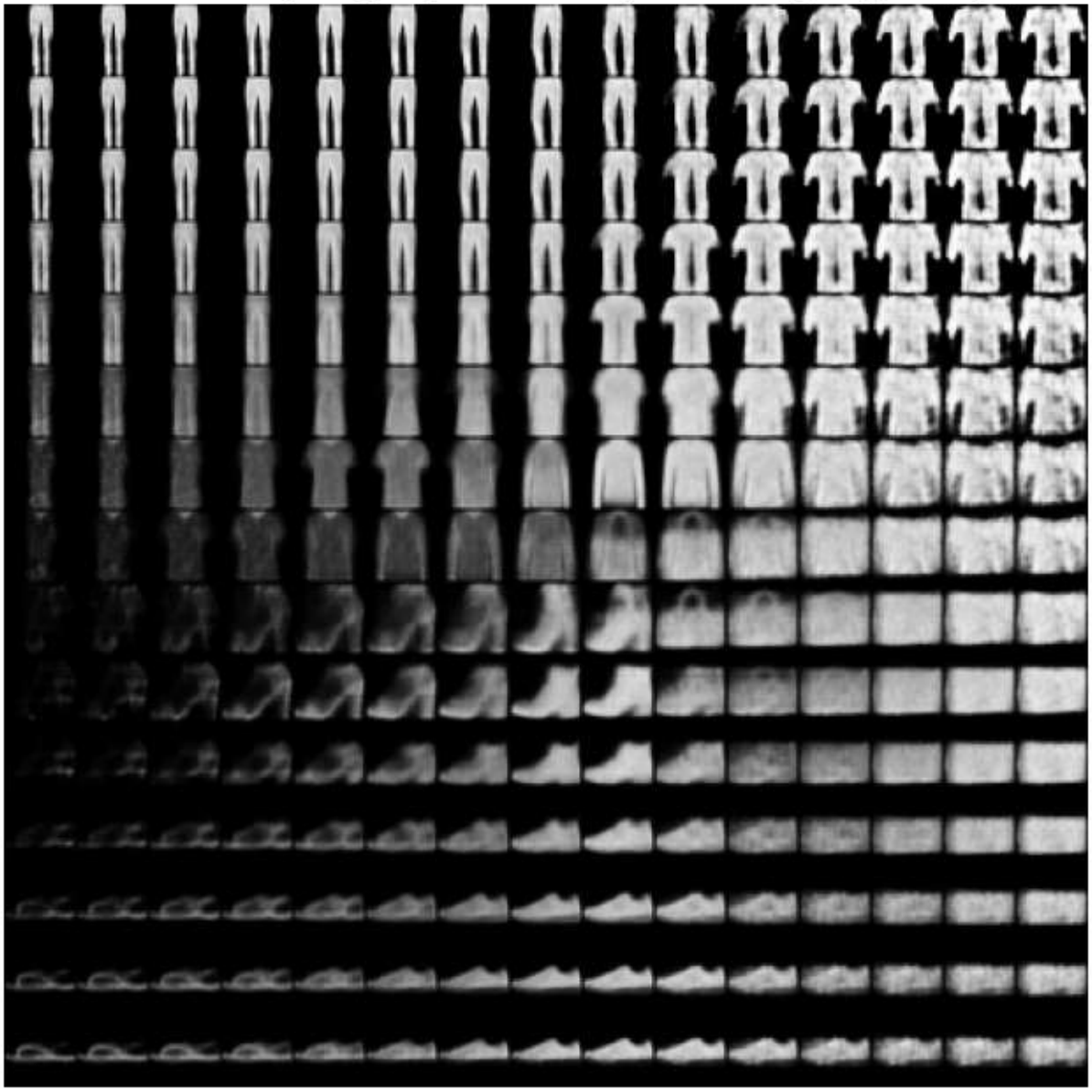
```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz
29515/29515 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz
26421880/26421880 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz
5148/5148 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz
4422102/4422102 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Training images: (60000, 28, 28, 1)
VAE epoch  1: total=320.77  reconstruction=315.70  KL=5.06
VAE epoch  2: total=273.29  reconstruction=267.39  KL=5.89
VAE epoch  3: total=269.47  reconstruction=263.49  KL=5.98
VAE epoch  4: total=267.47  reconstruction=261.44  KL=6.04
VAE epoch  5: total=266.28  reconstruction=260.20  KL=6.07
VAE epoch  6: total=265.32  reconstruction=259.22  KL=6.10
VAE epoch  7: total=264.46  reconstruction=258.33  KL=6.13
VAE epoch  8: total=263.83  reconstruction=257.66  KL=6.17
VAE epoch  9: total=263.42  reconstruction=257.21  KL=6.21
VAE epoch 10: total=262.88  reconstruction=256.68  KL=6.21
VAE epoch 11: total=262.41  reconstruction=256.17  KL=6.24
VAE epoch 12: total=262.03  reconstruction=255.75  KL=6.28
VAE epoch 13: total=261.78  reconstruction=255.51  KL=6.27
VAE epoch 14: total=261.45  reconstruction=255.13  KL=6.32
VAE epoch 15: total=261.03  reconstruction=254.73  KL=6.31
```



VAE: images generated from z ~ N(0, I)

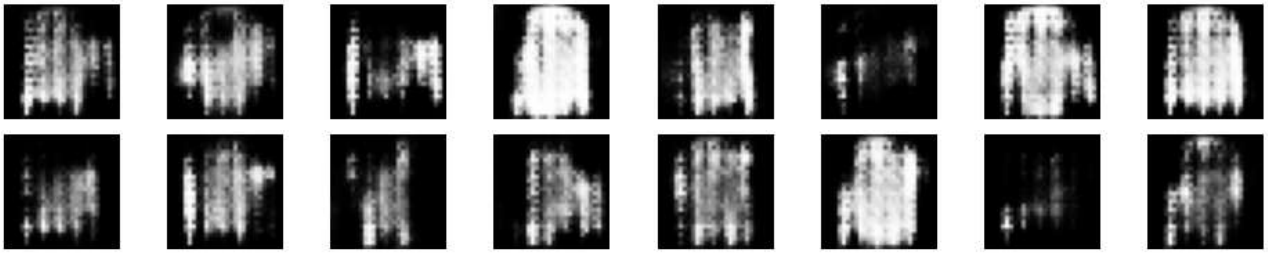


VAE latent-space grid (z1 horizontal, z2 vertical, range -3..3)



GAN epoch 1: generator loss=0.833 discriminator loss=1.190

GAN samples after epoch 1



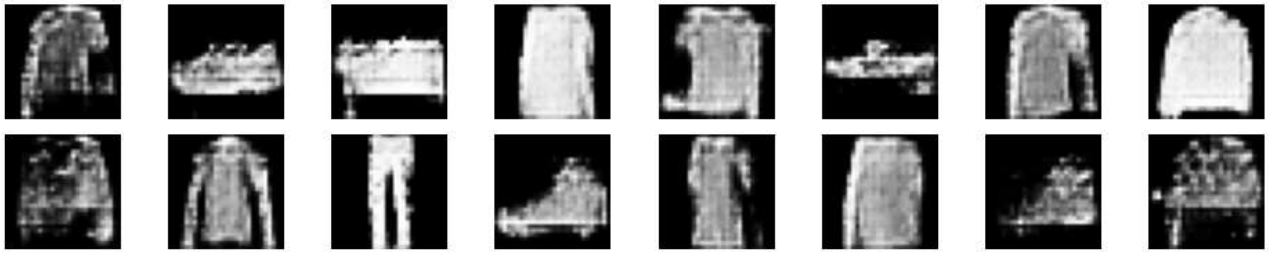
GAN epoch 2: generator loss=1.005 discriminator loss=1.086

GAN epoch 3: generator loss=1.108 discriminator loss=1.039

GAN epoch 4: generator loss=0.988 discriminator loss=1.153

GAN epoch 5: generator loss=0.944 discriminator loss=1.179

GAN samples after epoch 5



GAN epoch 6: generator loss=0.927 discriminator loss=1.191

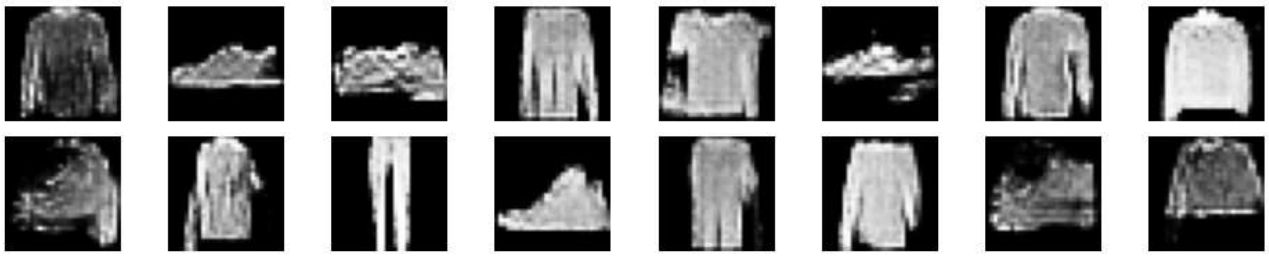
GAN epoch 7: generator loss=0.914 discriminator loss=1.208

GAN epoch 8: generator loss=0.902 discriminator loss=1.216

GAN epoch 9: generator loss=0.883 discriminator loss=1.229

GAN epoch 10: generator loss=0.877 discriminator loss=1.240

GAN samples after epoch 10



GAN epoch 11: generator loss=0.858 discriminator loss=1.257

GAN epoch 12: generator loss=0.856 discriminator loss=1.256

GAN epoch 13: generator loss=0.849 discriminator loss=1.262

GAN epoch 14: generator loss=0.854 discriminator loss=1.259

GAN epoch 15: generator loss=0.854 discriminator loss=1.261

GAN samples after epoch 15



GAN epoch 16: generator loss=0.854 discriminator loss=1.260

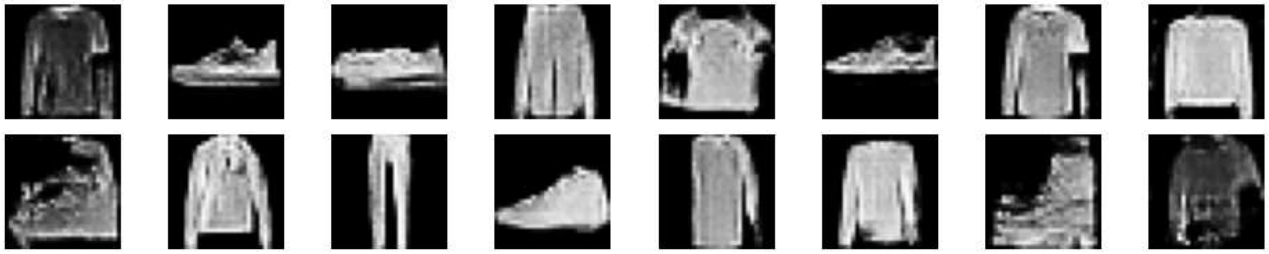
GAN epoch 17: generator loss=0.857 discriminator loss=1.259

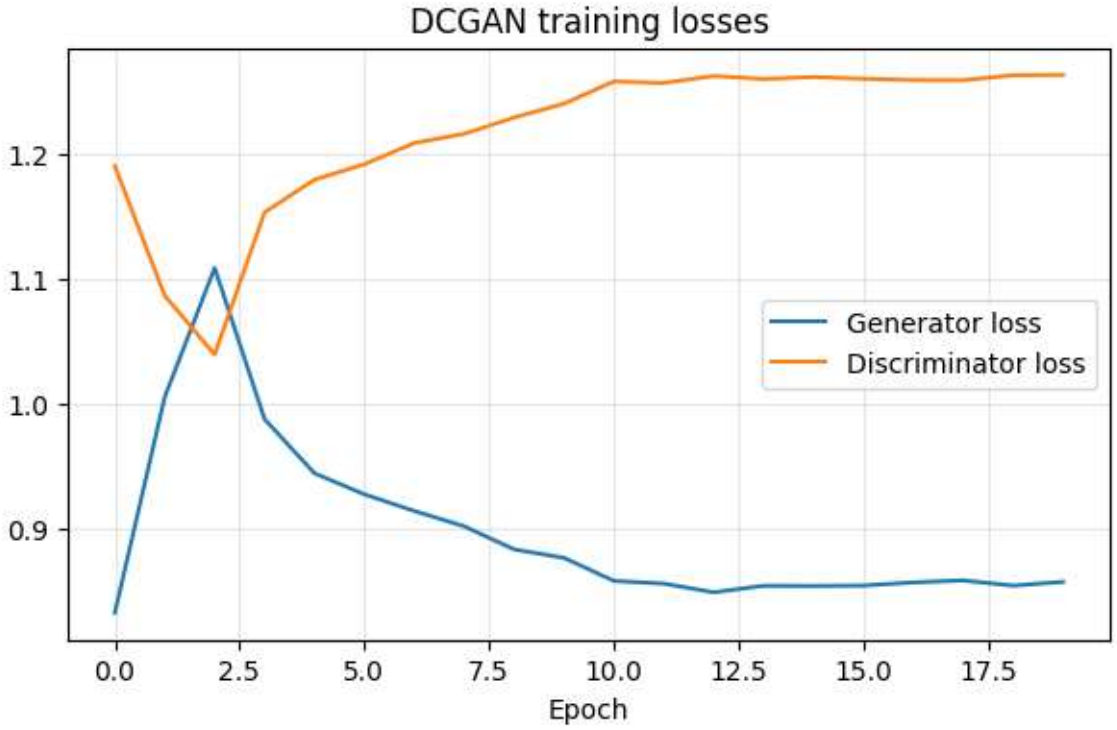
GAN epoch 18: generator loss=0.859 discriminator loss=1.258

GAN epoch 19: generator loss=0.854 discriminator loss=1.262

GAN epoch 20: generator loss=0.857 discriminator loss=1.263

GAN samples after epoch 20





Final DCGAN samples



Saved: vae_decoder.keras, gan_generator.keras and gan_samples/.png (Experiment 30 uses the two models)*